

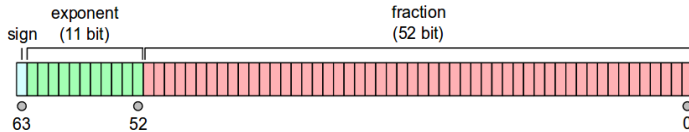
# Representación de números en punto flotante

Three horizontal bars of different colors (light green, medium green, and dark green) are stacked on top of each other, spanning the width of the slide.

Programación III - UNGS

# Punto flotante

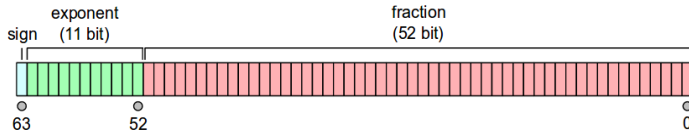
- Representamos **números reales** en Java por medio de **float** y **double**.





# Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.

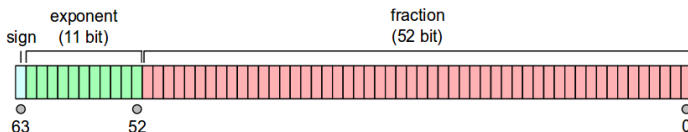


- La **representación de punto flotante** escribe un número  $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$ .



# Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.

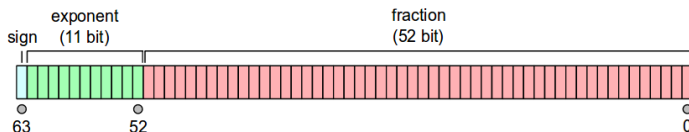


- La **representación de punto flotante** escribe un número  $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$ .
- El bit  $s$  es el **signo** de  $a$ .



# Punto flotante

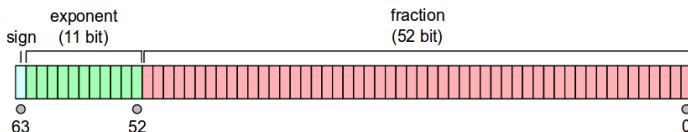
- Representamos **números reales** en Java por medio de **float** y **double**.



- La **representación de punto flotante** escribe un número  $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$ .
  - El bit  $s$  es el **signo** de  $a$ .
  - La secuencia  $a_1, \dots, a_n$  es la **mantisa**, y tiene  $n$  bits.

# Punto flotante

- Representamos **números reales** en Java por medio de **float** y **double**.



- La **representación de punto flotante** escribe un número  $a = (-1)^s 0.a_1a_2 \dots a_n \times 2^{e_1 \dots e_k}$ .
  - El bit  $s$  es el **signo** de  $a$ .
  - La secuencia  $a_1, \dots, a_n$  es la **mantisa**, y tiene  $n$  bits.
  - La secuencia  $e_1, \dots, e_k$  es el **exponente**, y tiene  $k$  bits.

# Punto flotante

---

- Esta representación tiene importantes consecuencias!

# Punto flotante

---

- Esta representación tiene importantes consecuencias!
  1. Como  $n$  y  $k$  son fijos, la cantidad de números representables por medio de **float** o **double** es **finita**.

# Punto flotante

---

- Esta representación tiene importantes consecuencias!
  1. Como  $n$  y  $k$  son fijos, la cantidad de números representables por medio de **float** o **double** es **finita**.
  2. La **distribución** de los números representables sobre la recta real no es **uniforme**. La representación es más densa cerca del cero.

# Punto flotante

---

- Esta representación tiene importantes consecuencias!
  1. Como  $n$  y  $k$  son fijos, la cantidad de números representables por medio de **float** o **double** es **finita**.
  2. La **distribución** de los números representables sobre la recta real no es **uniforme**. La representación es más densa cerca del cero.
  3. Al guardar un número real en una variable de tipo **float** o **double**, la representación puede no ser **exacta**.

# Punto flotante

---

- Esta representación tiene importantes consecuencias!
  1. Como  $n$  y  $k$  son fijos, la cantidad de números representables por medio de **float** o **double** es **finita**.
  2. La **distribución** de los números representables sobre la recta real no es **uniforme**. La representación es más densa cerca del cero.
  3. Al guardar un número real en una variable de tipo **float** o **double**, la representación puede no ser **exacta**.
  4. Al operar con números representados en punto flotante, el resultado puede tener **error de redondeo**.

# Punto flotante

---

- ¿Por qué existe entonces la representación en punto flotante?

# Punto flotante

---

- ¿Por qué existe entonces la representación en punto flotante?
  1. Es la mejor representación que tenemos para realizar cálculos con valores de magnitudes distintas.

# Punto flotante

---

- ¿Por qué existe entonces la representación en punto flotante?
  1. Es la mejor representación que tenemos para realizar **cálculos** con valores de magnitudes distintas.
  2. Los microprocesadores actuales manejan esta representación por **hardware**, y entonces las operaciones con **float** y **double** son muy eficientes.

# Punto flotante

---

- ¿Por qué existe entonces la representación en punto flotante?
  1. Es la mejor representación que tenemos para realizar **cálculos** con valores de magnitudes distintas.
  2. Los microprocesadores actuales manejan esta representación por **hardware**, y entonces las operaciones con **float** y **double** son muy eficientes.
- Sin embargo, el error de redondeo puede ser muy problemático si estamos en un contexto de **punto fijo** (por ejemplo, con cálculos monetarios) y necesitamos respuestas exactas.

# Punto flotante

---

```

1  int iteraciones = 0;
2  double fondos = 1;
3  double precio = 0.1;
4
5  while( fondos >= precio )
6  {
7      fondos -= precio;
8      precio += 0.1;
9      ++iteraciones;
10 }
11
12 System.out.println("Iteraciones: " + iteraciones);
13 System.out.println("Vuelto: $" + fondos);

```

---

# Punto flotante

---

```

1  int iteraciones = 0;
2  double fondos = 1;
3  double precio = 0.1;
4
5  while( fondos >= precio )
6  {
7      fondos -= precio;
8      precio += 0.1;
9      ++iteraciones;
10 }
11
12 System.out.println("Iteraciones: " + iteraciones);
13 System.out.println("Vuelto: $" + fondos);

```

---

- El resultado es bastante inesperado:

# Punto flotante

---

```

1  int iteraciones = 0;
2  double fondos = 1;
3  double precio = 0.1;
4
5  while( fondos >= precio )
6  {
7      fondos -= precio;
8      precio += 0.1;
9      ++iteraciones;
10 }
11
12 System.out.println("Iteraciones: " + iteraciones);
13 System.out.println("Vuelto: $" + fondos);

```

---

- El resultado es bastante inesperado:
- 

```

1  Iteraciones: 3
2  Vuelto: $0.3999999999999999

```

---

# Punto flotante

---

- **Conclusión.** La representación en punto flotante es inadecuada para realizar cálculos monetarios.

## Punto flotante

---

- **Conclusión.** La representación en punto flotante es inadecuada para realizar cálculos monetarios.
- En este caso, debemos usar representación de punto fijo.

# Punto flotante

---

- **Conclusión.** La representación en punto flotante es inadecuada para realizar cálculos monetarios.
- En este caso, debemos usar representación de punto fijo.
  1. En este caso, se fija la cantidad de decimales a la derecha del punto decimal.

## Punto flotante

---

- **Conclusión.** La representación en punto flotante es inadecuada para realizar cálculos monetarios.
- En este caso, debemos usar representación de punto fijo.
  1. En este caso, se fija la cantidad de decimales a la derecha del punto decimal.
  2. Las operaciones se realizan con la cantidad de decimales especificada, y los valores menos significativos se pierden.

# BigDecimal

---

- Las bibliotecas estándar de Java incluyen la clase **BigDecimal**, una implementación de números reales en punto fijo.

# BigDecimal

---

- Las bibliotecas estándar de Java incluyen la clase **BigDecimal**, una implementación de números reales en punto fijo.
- **BigDecimal** es un tipo de datos **immutable** (qué significa esto?) que implementa aritmética de punto fijo con la siguiente funcionalidad:

# BigDecimal

---

- Las bibliotecas estándar de Java incluyen la clase **BigDecimal**, una implementación de números reales en punto fijo.
- **BigDecimal** es un tipo de datos **immutable** (qué significa esto?) que implementa aritmética de punto fijo con la siguiente funcionalidad:
  1. Se puede especificar la cantidad de decimales a la derecha del punto decimal.

# BigDecimal

---

- Las bibliotecas estándar de Java incluyen la clase **BigDecimal**, una implementación de números reales en punto fijo.
- **BigDecimal** es un tipo de datos **inmutable** (qué significa esto?) que implementa aritmética de punto fijo con la siguiente funcionalidad:
  1. Se puede especificar la cantidad de decimales a la derecha del punto decimal.
  2. Se puede especificar el **método de redondeo** luego de cada operación.

# BigDecimal

---

- Las bibliotecas estándar de Java incluyen la clase **BigDecimal**, una implementación de números reales en punto fijo.
- **BigDecimal** es un tipo de datos **immutable** (qué significa esto?) que implementa aritmética de punto fijo con la siguiente funcionalidad:
  1. Se puede especificar la cantidad de decimales a la derecha del punto decimal.
  2. Se puede especificar el **método de redondeo** luego de cada operación.
- Creamos un **BigDecimal** así:

# BigDecimal

---

- Las bibliotecas estándar de Java incluyen la clase **BigDecimal**, una implementación de números reales en punto fijo.
  - **BigDecimal** es un tipo de datos **inmutable** (qué significa esto?) que implementa aritmética de punto fijo con la siguiente funcionalidad:
    1. Se puede especificar la cantidad de decimales a la derecha del punto decimal.
    2. Se puede especificar el **método de redondeo** luego de cada operación.
  - Creamos un **BigDecimal** así:
- 
- ```

1  BigDecimal big = new BigDecimal("81.9065");
2  BigDecimal big = BigDecimal.valueOf(81.9065);
    
```
-

# BigDecimal

---

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

# BigDecimal

---

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

- ```
1 big.setScale(2, RoundingMode.CEILING);
```

---

# BigDecimal

---

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

1 `big.setScale(2, RoundingMode.CEILING);`

---

- Si no se especifica la escala, entonces el valor usa la escala dada por el string de inicialización.

# BigDecimal

---

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

```
1 big.setScale(2, RoundingMode.CEILING);
```

---

- Si no se especifica la escala, entonces el valor usa la escala dada por el string de inicialización.
- Tenemos varias opciones:

# BigDecimal

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

```
1 big.setScale(2, RoundingMode.CEILING);
```

---

- Si no se especifica la escala, entonces el valor usa la escala dada por el string de inicialización.
- Tenemos varias opciones:
  - CEILING: Redondeo hacia  $+\infty$ .

# BigDecimal

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

```
1 big.setScale(2, RoundingMode.CEILING);
```

---

- Si no se especifica la escala, entonces el valor usa la escala dada por el string de inicialización.
- Tenemos varias opciones:
  - CEILING: Redondeo hacia  $+\infty$ .
  - DOWN: Redondeo hacia 0.

# BigDecimal

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

```
1 big.setScale(2, RoundingMode.CEILING);
```

---

- Si no se especifica la escala, entonces el valor usa la escala dada por el string de inicialización.
- Tenemos varias opciones:
  - CEILING: Redondeo hacia  $+\infty$ .
  - DOWN: Redondeo hacia 0.
  - FLOOR: Redondeo hacia  $-\infty$ .

# BigDecimal

- Podemos especificar la **precisión** de punto fijo y el método de redondeo:

---

```
1 big.setScale(2, RoundingMode.CEILING);
```

---

- Si no se especifica la escala, entonces el valor usa la escala dada por el string de inicialización.
- Tenemos varias opciones:
  - CEILING: Redondeo hacia  $+\infty$ .
  - DOWN: Redondeo hacia 0.
  - FLOOR: Redondeo hacia  $-\infty$ .
  - UP: Redondeo alejándose del 0.

# BigDecimal

---

```

1  int iteraciones = 0;
2  BigDecimal precio = new BigDecimal("0.10");
3  BigDecimal fondos = new BigDecimal("1.00");
4  BigDecimal diez_centavos = new BigDecimal( "0.10");
5
6  while( fondos.compareTo(precio) >= 0 )
7  {
8      fondos = fondos.subtract(precio);
9      precio = precio.add(diez_centavos);
10     ++iteraciones;
11 }
12
13 System.out.println("Iteraciones: " + iteraciones);
14 System.out.println("Vuelto: $" + fondos);

```

---

# BigDecimal

---

```

1  int iteraciones = 0;
2  BigDecimal precio = new BigDecimal("0.10");
3  BigDecimal fondos = new BigDecimal("1.00");
4  BigDecimal diez_centavos = new BigDecimal( "0.10");
5
6  while( fondos.compareTo(precio) >= 0 )
7  {
8      fondos = fondos.subtract(precio);
9      precio = precio.add(diez_centavos);
10     ++iteraciones;
11 }
12
13 System.out.println("Iteraciones: " + iteraciones);
14 System.out.println("Vuelto: $" + fondos);

```

---

- Tenemos ahora:

# BigDecimal

---

```

1  int iteraciones = 0;
2  BigDecimal precio = new BigDecimal("0.10");
3  BigDecimal fondos = new BigDecimal("1.00");
4  BigDecimal diez_centavos = new BigDecimal( "0.10");
5
6  while( fondos.compareTo(precio) >= 0 )
7  {
8      fondos = fondos.subtract(precio);
9      precio = precio.add(diez_centavos);
10     ++iteraciones;
11 }
12
13 System.out.println("Iteraciones: " + iteraciones);
14 System.out.println("Vuelto: $" + fondos);

```

---

- Tenemos ahora:

---

```

1  Iteraciones: 4
2  Vuelto: $0.00

```

---

# BigDecimal

---

- Entonces, ¿hay que usar siempre **BigDecimal** en lugar de **float** o **double**?

# BigDecimal

---

- Entonces, ¿hay que usar siempre **BigDecimal** en lugar de **float** o **double**?
- No! Depende del contexto.

# BigDecimal

---

- Entonces, ¿hay que usar siempre **BigDecimal** en lugar de **float** o **double**?
- No! Depende del contexto.
  1. **BigDecimal** es útil cuando la cantidad de decimales se puede prefijar de antemano.

# BigDecimal

---

- Entonces, ¿hay que usar siempre **BigDecimal** en lugar de **float** o **double**?
- No! Depende del contexto.
  1. **BigDecimal** es útil cuando la cantidad de decimales se puede prefijar de antemano.
  2. La aritmética de punto fijo tiene muchos inconvenientes si no se conoce de antemano el **rango** de los datos o resultados.

# BigDecimal

---

- Entonces, ¿hay que usar siempre **BigDecimal** en lugar de **float** o **double**?
- No! Depende del contexto.
  1. **BigDecimal** es útil cuando la cantidad de decimales se puede prefijar de antemano.
  2. La aritmética de punto fijo tiene muchos inconvenientes si no se conoce de antemano el **rango** de los datos o resultados.
  3. **BigDecimal** implementa una **emulación** de la aritmética, y es mucho menos eficiente que **float** y **double**.